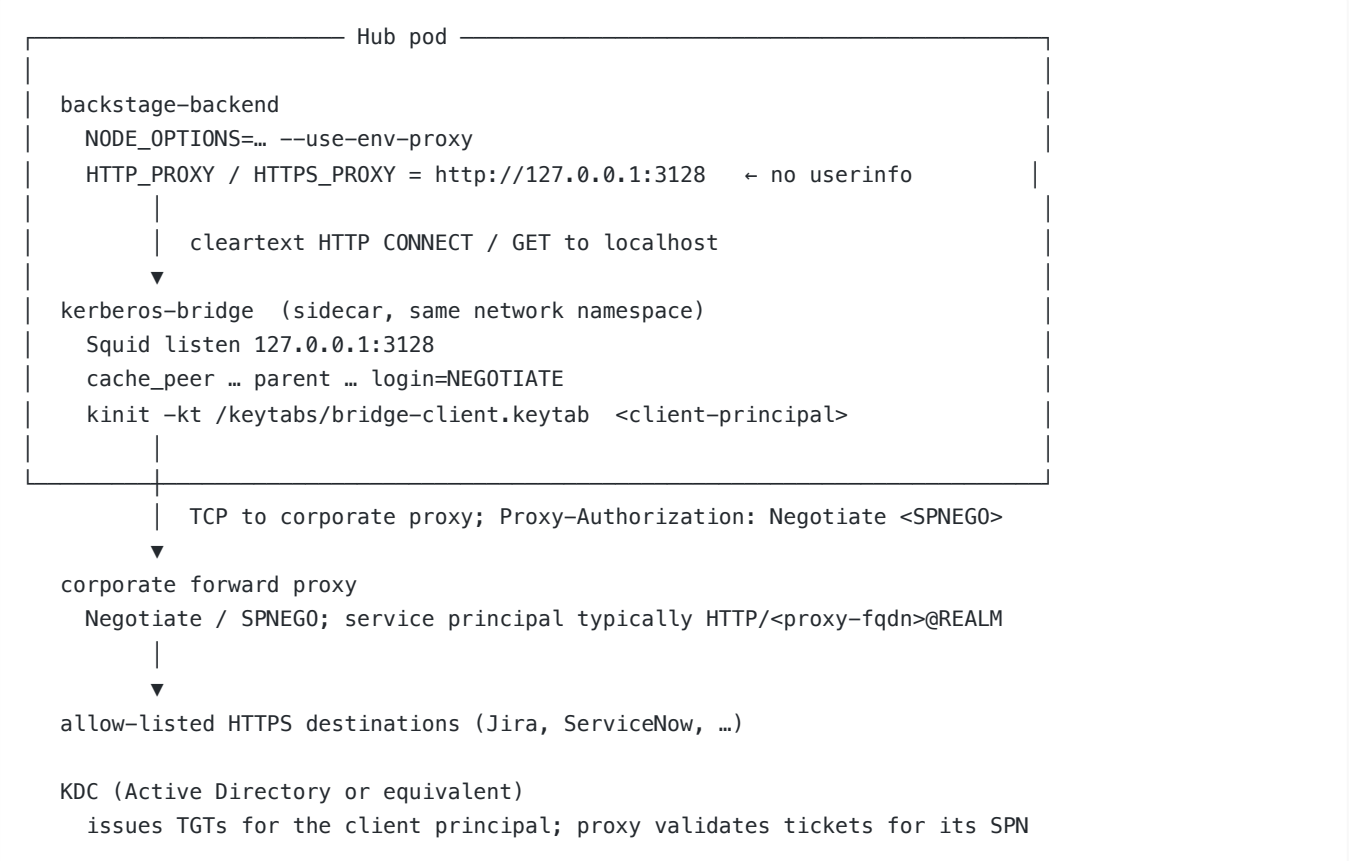
Negotiate bridge sidecar for Red Hat Developer Hub

When the corporate forward proxy requires **Kerberos Negotiate**, the Hub process cannot speak Negotiate natively. This pattern places a **localhost Squid sidecar** in the Hub pod: Hub uses `HTTP(S)_PROXY=http://127.0.0.1:3128` with **no proxy credentials**; the sidecar authenticates to the corporate proxy with **Negotiate** using a client keytab.

Image base: `registry.redhat.io/rhel9/squid` plus `krb5-workstation`. Example image tag: `registry.example.com/rhdh/kerberos-negotiate-bridge:0.1.0`.

---

1. End-to-end flow



Hop	Auth	Who authenticates
Hub → sidecar (`127.0.0.1:3128`)	**None**	Same pod / localhost only
Sidecar → corporate proxy	**Negotiate (Kerberos)**	Sidecar (`login=NEGOTIATE` + keytab)
Tunnel → SaaS	SaaS credential	Hub plugins, inside TLS after `CONNECT`

Hub never handles Kerberos. Plugins continue to use `fetch`; Node's `EnvHttpProxyAgent` only sees `HTTP(S)_PROXY`.

---

2. Why a sidecar (not a separate bridge Deployment)

Sidecar	Separate bridge Service
Hub uses `http://127.0.0.1:3128`	Hub uses `http://bridge.<ns>.svc:3128`
No-auth Squid is never a cluster Service	No-auth listener must be tightly NetworkPolicy-scoped

Keytab Secret mounts on the Hub pod	Keytab on a shared volume or separate pod
Shares Hub pod lifecycle	Independent scale / failure domain

Network policy should allow the Hub namespace to reach the corporate proxy VIP.

---

3. Sidecar container specification

Enabled via Hub chart values (`kerberosBridgeSidecar.enabled: true`) on the Backstage CR `deployment.patch` .

3.1 Identity

Field	Value
Container name	`kerberos-bridge`
Image	Negotiate bridge image (UBI/RHEL 9 Squid + `krb5-workstation`)
User	Non-root; under OpenShift `restricted-v2` do **not** hard-code `runAsUser: 1001` (UID must fall in the namespace range)
Capabilities	`drop: [ALL]`, `allowPrivilegeEscalation: false`, `seccompProfile: RuntimeDefault`

3.2 Listen port (Hub-facing)

Field	Requirement
`containerPort`	`3128` (name `proxy`)
Squid `http_port`	**`127.0.0.1:3128`** (loopback only). Must match Hub `HTTP_PROXY`.
Client auth on listen	**None**. Safe **only** with the loopback bind.
Probes	**exec** `bash -c 'echo >/dev/tcp/127.0.0.1/3128'` (not `tcpSocket` to the pod IP)

3.3 Volume: client keytab

Field	Requirement
Volume	Secret (e.g. `kerberos-bridge-keytab`)
Mount	`/keytabs` read-only
File	`/keytabs/bridge-client.keytab`
Mode	Readable by the SCC-assigned UID (e.g. `defaultMode: 0444`)

Without this file the entrypoint exits and Squid never listens.

3.4 Environment (sidecar)

Env	Purpose	Example
`REALM`	Kerberos realm	`EXAMPLE.COM`
`KDC_HOSTNAME`	KDC for `kinit` / `krb5.conf`	`kdc.example.com`
`CORPORATE_PROXY_HOSTNAME`	Squid `cache_peer` parent host	`proxy.example.com`

`CORPORATE_PROXY_PORT`	Parent listen port	`3128`
`BRIDGE_CLIENT_PRINCIPAL`	Client principal in the keytab	`rhdh-bridge@EXAMPLE.COM`
`KRB5_KTNAME`	Keytab path	`/keytabs/bridge-client.keytab`
`HOME`	Writable home for the credential cache	`/tmp`

3.5 Squid behaviour (parent / corporate hop)

```
http_port 127.0.0.1:3128
http_access allow all
cache_peer <CORP_HOST> parent <CORP_PORT> 0 no-query default login=NEGOTIATE
never_direct allow all
cache deny all
```

Directive	Meaning
`login=NEGOTIATE`	Squid obtains a service ticket and sends `Proxy-Authorization: Negotiate …`
`never_direct allow all`	All traffic goes to the parent proxy
`cache deny all`	Forward-only

Entrypoint:

1. Write `/tmp/krb5.conf` with an explicit KDC host.
2. `kinit -kt` with retries.
3. Periodic ticket refresh.
4. `squid -N` with the rendered config.

---

4. Requirements to reach the corporate proxy

4.1 Kerberos client identity (sidecar)

Requirement	Detail
Client principal	Principal allowed to authenticate **to** the proxy as a client
Keytab	Keytab for that principal only (not the proxy server keytab)
Reachable KDC	Kerberos ports from the Hub pod network to the KDC
Clock sync	Normal Kerberos skew tolerance (cluster NTP)
Ticket lifetime	Sidecar renews via periodic `kinit`; align with realm policy

4.2 Corporate proxy

Requirement	Detail
Negotiate enabled	Proxy challenges with Negotiate / accepts SPNEGO
Service principal (SPN)	Typically `HTTP/<proxy-fqdn>@REALM` matching the hostname the **sidecar dials**
Server keytab on proxy	Proxy process holds that SPN's keys
Allow list	Destinations Hub will `CONNECT` to (SaaS hosts)

Network path	Hub namespace → proxy host:port

If the FQDN the sidecar dials does not match the SPN host part, Negotiate fails even with a valid client ticket.

4.3 Environment mapping

Concept	Typical enterprise equivalent
KDC	Active Directory
Client principal + keytab	App account / gMSA / IAM-issued keytab
Corporate proxy	Existing forward proxy with Kerberos
`HTTP/<proxy-fqdn>@REALM`	SPN registered in AD for the proxy hostname
Allow list	Proxy policy for SaaS egress
Network path	Firewall / NSG / NetworkPolicy to the proxy VIP

SaaS credentials remain in Hub Secrets as today; they are not used for proxy Negotiate.

---

5. Hub container settings

Setting	Value
`HTTP_PROXY`	`http://127.0.0.1:3128` (**no** `user:pass@`)
`HTTPS_PROXY`	Same URL
`NO_PROXY`	In-cluster and registry hosts as required; omit SaaS hosts that must use the proxy
`NODE_OPTIONS`	`--no-node-snapshot --use-env-proxy`
Plugin-install init	Keep off the Hub proxy path

---

6. Example Helm values

```yaml
httpProxy:
  enabled: true
  nodeOptions: "--no-node-snapshot --use-env-proxy"

kerberosBridgeSidecar:
  enabled: true
  image: registry.example.com/rhdh/kerberos-negotiate-bridge:0.1.0
  keytabSecret: kerberos-bridge-keytab
  port: 3128
  realm: EXAMPLE.COM
  kdcHostname: kdc.example.com
  corporateProxyHostname: proxy.example.com
  corporateProxyPort: "3128"
  clientPrincipal: rhdh-bridge@EXAMPLE.COM
```

---

7. Operational checklist

1. KDC and corporate proxy reachable; client keytab available as a Secret in the Hub namespace.
2. Image pullable in the Hub namespace.
3. Hub `HTTP_PROXY` / `HTTPS_PROXY` set to `http://127.0.0.1:3128`.
4. Hub pod includes container `kerberos-bridge`; readiness uses the loopback exec probe.
5. Smoke test from the Hub container:

```
oc -n <hub-namespace> exec deploy/<backstage-deploy> -c backstage-backend -- \
  sh -c 'echo "$HTTPS_PROXY"; node --use-env-proxy -e
"fetch(\"https://example.com\").then(r=>console.log(r.status))"'
```

Expect `http://127.0.0.1:3128` and HTTP **200** for an allow-listed host.

Failure modes

Symptom	Likely cause
Sidecar not Ready; logs `keytab missing`	Secret not mounted / wrong key name
Sidecar loops waiting for KDC	Network to KDC, wrong `KDC_HOSTNAME` / realm
`kinit` OK but Hub gets 407/502 via proxy	SPN mismatch, Negotiate misconfig, allow-list deny
Hub `ENOTFOUND bridge.…`	Secret still points at a Service URL — use localhost
Pod `FailedCreate` / SCC `runAsUser: 1001`	Remove fixed UID; use the namespace SCC range

---

8. Related chart paths

Path	Role
`charts/kerberos-lab/images/bridge-ubi/`	Squid + Kerberos image sources
`charts/developer-hub/templates/backstage.yaml`	Sidecar on the Backstage CR
`charts/developer-hub/values.yaml`	`kerberosBridgeSidecar.*` knobs